Password Change on First Sign-in and Admin-Handled Password Reset: Architecture and Implementation Plan

Project: Cashflow application, ADT Dental Clinic
Date: November 5, 2026 (revised from the November 4 version)
Status: Built and tested on November 5, 2026 (see "As built" at the end). Needs the two SQL files run in Neon before it is active.
Scope (as requested): (1) a newly created account must choose its own password before it can open any other page; (2) a forgotten password is handled by an admin, who can answer a request or reset a password manually at any time; (3) forgot password leads back into the same change-password flow; (4) there is exactly one **superuser**, who has access to everything.
Related: `PERMISSIONS_ARCHITECTURE.md` (the guard module this plan builds on) and item 2 of `OPTIMIZATION_SECURITY_REVIEW.md` (change-password and the placeholder forgot-password page).

---

1. Summary

Today an admin types a password when creating a user and that password stays forever; nobody can change their own password; the "Forgot password" page only pretends to send an e-mail. This plan replaces that with two connected flows that need **no e-mail service**:

1. **Temporary password, forced change.** A new account (and any account an admin resets) gets a one-time **temporary password** shown once to the admin. The account carries a flag `must_change_password`. While the flag is on, the only thing the person can do after signing in is open the **Change password** page (or sign out). Once they choose their own password the flag clears and the app opens normally.
2. **Forgot password through the admin.** The person asks from the sign-in page; the request appears on the admin's **Users** page (with a count in the menu). The admin verifies the person, presses **Reset password**, and hands over a new temporary password. The admin can also reset any password at any time from the Users list without waiting for a request. The reset puts the person back into flow 1.

Revision (Nov 5). The sign-in page's **Forgot password** now has three ways to a new password, all ending in the same forced change (section 4.2): the person **requests** a reset, an **admin resets manually** for a person who triggered a request, or an admin resets **at any time** without a request. A fourth, signed-in way is the voluntary change on the Profile page. The plan also adds the one-person **superuser** role (section 5A), which decides who may reset whom.

The two flows share one mechanism (the flag), so there is one rule to enforce and one page to build. The rule is enforced in **three layers** (the proxy, a per-request check in every page, action and route, and a short-lived token flag), because a check in only one place can be skipped, as section 5 explains. Changing or resetting a password also **signs the person out everywhere else**.

Estimated effort: about **6 working days**, in four phases (section 8). Phases 1 and 2 already deliver the first request.

---

2. What exists today

Item	Today
Creating a user	`createUserAction` (admin only): the admin types name, e-mail, role, branch **and the password** (min 8 characters). Stored as a bcrypt hash (`users.password_hash`).
Changing a password	**Not possible** for anyone, including admins. The Profile page has branch and theme only.
Forgot password	`/forgot-password` shows "If an account exists you'll receive reset instructions", but the code is a `setTimeout` placeholder. Nothing is sent or recorded. The page and its hero text also promise "reset links".
Get access page	Says the admin sends a temporary password, which is the process this plan formalizes.
Sign-in protection	Lock after 5 wrong passwords per e-mail in 15 minutes, a `login_attempts` table, a decoy hash for unknown e-mails (already built).
Session	JWT, 30-minute idle limit, 12-hour absolute limit. The role is read into the token at sign-in and refreshed on "Stay signed in".
Where the sign-in check happens	`proxy.ts` checks only "signed in or not" for `/cashflow/*`. Each page and action then calls `auth()` itself. API routes under `/api/cashflow/*` are outside the proxy.

---

3. Design principles

1. **The database is the truth, the token is a hint.** The flag lives on the user row. The token copy only lets the proxy redirect early without a database call.
2. **Enforce on the server, on every path.** A person who must change their password must not be able to reach data through a page URL, a direct action call, or an API route.
3. **An admin never chooses someone's real password.** The admin only issues a one-time temporary password, which the system generates, which expires, and which the person must replace.
4. **A request grants nothing.** Asking for a reset changes no account. Only an admin action does, after checking who is asking.
5. **Same answer for everyone.** The request form never reveals whether an e-mail has an account.
6. **Leave a trail, never a secret.** Every creation, reset, change and request is audited; passwords (temporary or real) are never logged or stored in readable form.
7. **A reset ends old sessions.** If someone else had the old password, resetting it must also end their sign-in.

---

4. Flows

4.1 New account: first sign-in

```
Admin: Users -> Add user (name, e-mail, role, branch)
   system generates temporary password, stores only its hash,
   sets must_change_password = true, temp_password_expires_at = now + 3 days
   screen shows the temporary password ONCE (copy button + a ready-to-send message)
        |
Admin gives it to the person (in person, phone, chat: not by e-mail)
```

```
        |
Person signs in at /login with e-mail + temporary password
   wrong password / locked           -> existing messages
   temporary password expired         -> "This temporary password has expired. Ask your admin fo
r a new one."
   correct                            -> session created with mustChangePassword = true
        |
Proxy: flagged + anywhere under /cashflow -> redirect to /change-password
        |
/change-password  ("Welcome, <name>. Choose your own password to continue.")
   current (temporary) password, new password, confirm new password
   rules shown live; sign-out link; same 30-minute idle rule
   server: verify current, check rules, save hash,
           must_change_password = false, password_changed_at = now,
           temp_password_expires_at = null, session_version + 1,
           audit "password changed", refresh the token
        |
Redirect to /cashflow (or the page they first asked for)
```

4.2 Forgot password: request, then the admin resets

Way in	Who starts it	Who acts	Result
A. Request	The person, from **Forgot password?** on the sign-in page	An admin (superuser for admin accounts), from the Password requests panel	Temporary password, forced change
B. Manual, because of a request	The person has already asked (or told the admin in person)	The admin presses **Reset password** on that person's request row	Same; the request is marked Done
C. Manual, no request	The admin	The admin presses **Reset password** on the Users list	Same
D. Voluntary	The signed-in person, from Profile	The person	New password right away; no temporary password involved

Ways A to C all end in the same place: the person gets a temporary password and the **change-password flow runs again** (flag set, nothing else reachable until they choose their own password).

```
Person: /login -> "Forgot password?" -> /forgot-password
   enters e-mail (optional note: "I'm at Marikina, phone 09...")
   server: stores a request (see 6.2); ALWAYS answers
   "Request sent. An admin will contact you with a temporary password.
    If it is urgent, ask your branch admin directly."
        |
Admin: sees "Password requests (2)" on Users, and a count beside Users in the menu
   row: name, e-mail, branch, requested at, note, browser / address summary
        |
Admin verifies the person (in person, or a call back to a known number)
        |
Admin: Reset password -> confirm
```

```
   system generates a temporary password, stores only its hash,
   must_change_password = true, expires in 3 days,
   session_version + 1 (ends any current sign-in of that user),
   clears that e-mail's sign-in lock, marks the request Done (who, when),
   audit "password reset by <admin>"
   screen shows the temporary password ONCE (copy + ready-to-send message)
        |
Admin gives it to the person -> person signs in -> forced change (flow 4.1)
```

An admin can also press **Dismiss** on a request (wrong person, duplicate, typo). Dismissed and done requests stay in a short history for audit.

4.3 Manual reset by an admin (no request)

Same as 4.2 from "Reset password": a button on each row of the Users list. The request step is optional. Used when someone tells the admin in person that they forgot, when an account is locked out, or when an admin suspects a password was seen by someone else.

4.4 Voluntary change later

Profile gets a **Password** section (current, new, confirm) using the same action and rules. It does not set the flag, and it also ends the person's other sign-ins.

---

5. Enforcement: why three layers

A common mistake is to check the flag in the page layout only. In the Next.js App Router, a layout is not re-run on every client-side navigation, and server actions and API routes never go through a layout, so a flagged person could still call them. The plan therefore checks in three places:

Layer	Where	What it does	Why it is not enough alone
1. Proxy (`proxy.ts`)	Every request to `/cashflow/*` (pages, navigation requests and action POSTs to those URLs)	If the token says `mustChangePassword`, redirect pages to `/change-password`; answer other requests with 403. Fast, no database.	The token can be stale for a moment; API routes under `/api/cashflow/*` and actions posted to other URLs are outside this path.
2. Per-request guard	`getActor()` / `requirePermission()` from the permissions plan, called at the top of every page, server action and API route	Reads the user row (`must_change_password`, `session_version`) once per request. Flagged: pages redirect to `/change-password`, actions return "Change your password first.", routes return 403. A `session_version` that no longer matches the token signs the person out.	This is the authoritative check. It must be present everywhere, which the build-time coverage check from the permissions plan enforces.
3. Token flag	`token.mustChangePassword`, `token.sessionVersion`	Carried in the sign-in token so layer 1 needs no database. Updated after a change through the existing "update" refresh.	A hint only; the database always wins.

Allowed while flagged

Path	Allowed	Why
`/change-password` and its action	Yes	The one thing the person must do
Sign out	Yes	They may leave
`/api/auth/*` (sign-in, session, sign-out)	Yes	The sign-in mechanism itself
`/privacy`, `/terms`, `/help`, `/contact`, `/api/health`	Yes	Public pages, no data
`/login`, `/forgot-password`	Redirect to `/change-password` if signed in	Same as other signed-in people
Everything under `/cashflow/*`, `/api/cashflow/*`, every other server action	**No**	

Dependency. Layer 2 is the guard module from `PERMISSIONS_ARCHITECTURE.md`. If this work is built first, Phase 1 introduces a minimal `requireActor()` in the same place and the permissions work extends it, so nothing is built twice. The twelve action files that today each have their own `requireSession` helper are changed to call it.

Why `session_version`. The app uses a signed token (no server-side session list), so a person cannot be "logged out remotely" by deleting a row. Instead each user has a counter. Changing or resetting a password adds 1. The token remembers the counter it was issued with; a mismatch on the next request (layer 2) ends that session. This is also what makes an admin reset effective when someone else knows the old password.

5A. The superuser and who may reset whom

The clinic has **exactly one superuser**: one person who can do everything, including managing admins. This is a new fourth role next to `admin`, `dentist` and `staff`. It also fits the permission table in `PERMISSIONS_ARCHITECTURE.md`, where `superuser` simply holds every permission (so new features never need a special case for them).

Keeping it to one person

- A partial unique index in the database (`one row where role = 'superuser'`) makes a second superuser impossible, even through a bug.
- The app never creates, demotes, promotes to, or deletes the superuser. The **Add user** and **Change role** lists do not offer "Superuser", and the actions refuse it server-side.
- The superuser account is made once, by SQL (section 6.3), from the existing test account or from a new one. Handing the role to another person later is a deliberate, rare change done by the developer with one SQL statement, not a button.

Who may reset whose password (checked in the server action, not only hidden in the screen)

Acting person	May reset	May not reset
Superuser	Anyone except themselves	Themselves (use Profile > Change password)
Admin	Dentists and staff	Other admins, the superuser, themselves
Dentist, staff	Nobody	

- A request for an **admin** account appears for admins as read-only ("Only the superuser can reset this account") and as actionable for the superuser. This replaces the earlier idea of an admin re-entering their own password to reset another admin: now only the superuser can, which is simpler and stricter.
- A request typed for the **superuser's** e-mail gets the same on-screen answer as any other, is not shown to admins, and is only recorded in the audit trail. Because there is no one above the superuser, a forgotten superuser password is handled by the developer's break-glass script (section 9.3), which should be protected with the same care as the database connection.
- The superuser is **never forced** to change their password by someone else's action, since nobody can reset it for them. Its own first password change, if wanted, is set once with the setup SQL (D9).
- The superuser can also see and act on every open request, which makes them the fallback when no admin is available.

---

6. Data model

All additive; existing users keep working and are **not** forced to change anything unless the clinic chooses to (decision D6).

6.1 `users` (4 new columns)

Column	Type	Meaning
`must_change_password`	boolean, default false	True after creation and after an admin reset
`password_changed_at`	timestamptz, null	When the person last chose their own password (null: still the original)
`temp_password_expires_at`	timestamptz, null	When the temporary password stops working (null: no temporary password)
`session_version`	integer, default 0	Raised by one on every password change or reset

6.2 `password_reset_requests` (new table)

Column	Type	Meaning
`id`	uuid	
`user_id`	uuid, null, references users, on delete set null	The matching account; null when no account has that e-mail
`email_key`	text	The typed e-mail, trimmed and lower-cased
`note`	text	What the person typed (limited to 300 characters)
`ip`, `user_agent`	text	For the admin's "is this plausible" check
`status`	`open`, `done` or `dismissed`	
`created_at`	timestamptz	
`handled_by`	uuid, null	The admin
`handled_at`	timestamptz, null	

Index on `(status, created_at desc)` and on `(email_key, created_at)`. Requests for unknown e-mails are stored with `user_id` null and shown to the admin as "no matching account" (often a typo); they are capped and purged after 30 days (D4).

6.3 SQL file (written for you to run in the Neon SQL editor)

`scripts/sql/2026-11-password-management.sql`, safe to repeat:

```
        alter table users add column if not exists must_change_password boolean not null defaul
t false;
        alter table users add column if not exists password_changed_at timestamptz;
        alter table users add column if not exists temp_password_expires_at timestamptz;
        alter table users add column if not exists session_version integer not null default 0;

        create table if not exists password_reset_requests (
          id uuid primary key default gen_random_uuid(),
          user_id uuid references users(id) on delete set null,
          email_key text not null,
          note text not null default '',
          ip text not null default '',
          user_agent text not null default '',
          status text not null default 'open',
          created_at timestamptz not null default now(),
          handled_by uuid references users(id) on delete set null,
          handled_at timestamptz
        );
        create index if not exists password_reset_requests_status_idx on password_reset_requests
(status, created_at desc);
        create index if not exists password_reset_requests_email_idx on password_reset_requests
(email_key, created_at);
```

Superuser setup: a second, separate run. Postgres does not let a new enum value be used in the same run that adds it, so this goes in `scripts/sql/2026-11-superuser-role.sql` and is run **after** the one above has finished:

```
        -- Run 1 (by itself)
        alter type user_role add value if not exists 'superuser';

        -- Run 2 (after run 1 has finished)
        create unique index if not exists users_single_superuser on users (role) where role = 's
uperuser';
        update users set role = 'superuser' where email = '<the superuser e-mail>' and not exist
s (select 1 from users where role = 'superuser');
```

The `update` only runs when no superuser exists yet. Which e-mail becomes the superuser is decision D9.

Run order. Run this SQL **before** the new code starts. The sign-in reads the whole `users` row, so without the columns sign-in would fail; as with earlier changes, the sign-in query also falls back safely (no flag) if the columns are missing, and prints one warning.

---

7. Code architecture

7.1 New and changed files

File	Change
`src/lib/auth/password-policy.ts` (new, pure)	`validateNewPassword(password, { email, name, current })` returning a message or null; `generateTempPassword()`; `TEMP_PASSWORD_DAYS`; a small blocklist of the most common passwords.
`src/lib/db/users.ts`	`getUserByEmail` selects the new columns; new `getAuthState(id)`, `setPassword({ id, hash, mustChange, expiresAt })`, `bumpSessionVersion(id)`.
`src/lib/db/password-requests.ts` (new)	`createRequest`, `listOpenRequests`, `countOpenRequests`, `resolveRequest`, `dismissRequest`, `purgeOld`.
`src/lib/db/schema.ts`	New columns and table.
`src/auth.ts`	`authorize` refuses an expired temporary password (code `temp_expired`) and returns the flag; `jwt` stores `mustChangePassword` and `sessionVersion` and re-reads them on the "update" refresh; `session` exposes `user.mustChangePassword`.
`src/types/next-auth.d.ts`	The two new fields.
`src/proxy.ts`	Layer 1 (section 5): redirect or 403 while flagged; add `/change-password` and `/forgot-password` to the matcher.
`src/lib/auth/actor.ts` and `guards.ts` (from the permissions plan)	Layer 2: refuse a flagged actor; compare `session_version`.
`src/app/change-password/page.tsx`, `actions.ts` (new)	The page (uses the sign-in layout, no menu) and `changePasswordAction`.
`src/components/auth/change-password-form.tsx` (new)	Form with show / hide, live rules, confirm; reused on Profile.
`src/app/cashflow/profile/page.tsx`	Password section.
`src/app/forgot-password/page.tsx`, `forgot-password-form.tsx`	Real request (`requestPasswordResetAction`), honest wording.
`src/app/cashflow/users/actions.ts`	`createUserAction` generates the temporary password (and refuses the `superuser` role); new `resetUserPasswordAction` (with the who-may-reset-whom rule), `dismissResetRequestAction`.
`src/lib/cashflow/user-schema.ts`, `src/lib/db/schema.ts`	`superuser` added to the role list (and to the database enum); role pickers exclude it.
`src/lib/auth/reset-rules.ts` (new, pure)	`canResetPassword(actor, target)`: the table in section 5A, easy to test.
`src/components/cashflow/users-manager.tsx`	"Password requests" panel, **Reset password** button per row, one-time password dialog, "Add user" without a password field.
`src/components/cashflow/cashflow-nav.tsx`, `cashflow/layout.tsx`	Count of open requests beside **Users** (admins only).

File	Change
`src/lib/cashflow/audit-types.ts`, `src/lib/db/audit.ts`	Actions `user.created`, `user.password_reset`, `auth.password_changed`, `auth.reset_requested`; entity type `user`.
`scripts/reset-password.ts` (new)	Break-glass for the developer (section 9.3).
`scripts/sql/2026-11-password-management.sql` (new)	Section 6.3.
Copy changes	Get-access page and the sign-in lock message ("Forgot your password? Request a reset.")

7.2 Password rules (`password-policy.ts`)

Rule	Value
Minimum length	**10** characters (today 8; D3)
Maximum	72 **bytes** in UTF-8 (the limit of the bcrypt hashing used; longer input would be silently cut)
Must differ from	the temporary / current password, the e-mail address, the person's name
Common passwords	refused against a built-in list of the most common ones (for example `password123`, the clinic's name)
Spaces	allowed (passphrases are encouraged); no forced symbol or case rules
Confirmation	typed twice

Temporary password. 12 characters from an alphabet without look-alike characters (no `0 O 1 I l`), drawn with Node's `crypto.randomInt` (about 59 bits), shown in groups ("K7M3-QX9P-RD4T"). Expires in **3 days** (D2); an expired one cannot sign in, and the admin issues a new one.

7.3 Server actions

`changePasswordAction(current, next, confirm)` (any signed-in person, including flagged)

1. Actor from the session; load the user row.
2. Count wrong current-password attempts with the same lock rule as sign-in (5 in 15 minutes), recorded in `login_attempts` as `wrong_current_password`, so a stolen session cannot be used to guess the password.
3. Verify the current password; validate the new one (7.2).
4. Save the hash; `must_change_password = false`; `password_changed_at = now`; clear the expiry; `session_version + 1`.
5. Audit `auth.password_changed` (no password in the log).
6. Refresh the token (`unstable_update`) so the new `sessionVersion` and cleared flag are in the cookie; redirect.

`resetUserPasswordAction(userId, requestId?)` (admin or superuser, permission `user.reset_password`)

1. Refuse for your own account ("Use Change password on your profile").
2. Apply the table in section 5A on the **server**: an admin may reset only dentists and staff; only the superuser may reset an admin; nobody may reset the superuser.
3. Generate the temporary password; save its hash; set the flag, the 3-day expiry, `session_version + 1`.

4. Clear that e-mail's sign-in lock; mark the request Done (who, when).
5. Audit `user.password_reset` (who reset whom); return the temporary password **once** in the response.

`createUserAction`: same generation; audit `user.created`. The Add user form loses its password field.

`requestPasswordResetAction(email, note)` (public, no sign-in)

- Always returns the same message.
- At most **one open request per e-mail**; a repeat updates its time instead of adding a row.
- At most **5 requests per hour per address** (reusing the `login_attempts` approach), a 300-character note limit, and a cap on stored unknown-e-mail requests.
- Audit `auth.reset_requested` only when an account matched.

7.4 What the admin sees

- **Users page, "Password requests"** panel above the list: open requests, newest first, with name, e-mail, branch, time, note, a "no matching account" tag where it applies, **Reset password** and **Dismiss**. Hidden when empty.
- **Menu:** a small count beside **Users** for admins (one `count(*)` on open requests in the layout, admins only, or added to the existing open-visits poll).
- **One-time password dialog** after create or reset: the temporary password in large type, a **Copy** button, the expiry date, and a **Copy message** button with ready text, for example: *"Hi Maria, your temporary Cashflow password is K7M3-QX9P-RD4T. Sign in at (address) and you will be asked to choose your own password. It expires on Nov 7."* A reminder: *confirm it is really them before you hand it over; do not send it by e-mail.* Closing the dialog loses the password; to get another, reset again.
- **Users list:** a "Must change password" tag and "Temporary password expires ..." beside each flagged person.

---

8. Rollout plan

Estimates are rough working days for one developer including checks.

Phase	What	Files	Est.
0. Foundations	SQL file, schema columns and table, `password-policy.ts` with tests, audit action types, token and session types	schema, SQL, policy, types	0.5
1. Forced change	`auth.ts` token fields and expiry rule; proxy gate; `getActor` / guard check (minimal `requireActor` if the permissions work is not in yet) and the same check in the 12 action files and 5 API routes; `/change-password` page and action; Profile password section	auth, proxy, guards, change-password, profile	2
2. Admin-issued passwords	Generated temporary passwords on **Add user**; **Reset password** per row with the one-time dialog; session version; unlock on reset; admin re-entry when resetting an admin; audit lines	users actions and manager	1.5
3. Forgot password through the admin	Real request form and honest text; request table and throttles; admin panel; menu count; dismiss	forgot-password, users, nav	1.5
4. Hardening and tests	Tests (section 10), break-glass script, copy changes, the coverage check extended to "every action honors the flag", a one-time "force	scripts, tests	0.5

Phase	What	Files	Est.
	change for existing users" query if chosen (D6)		

Total about **6 days**. Ship phases 0 to 2 together (new accounts and admin resets work end to end); phase 3 follows.

Going live safely. Run the SQL; deploy; keep one admin signed in on a second device; test with a throw-away account first. If anything goes wrong the flag can be cleared with one SQL line (`update users set must_change_password = false where email = '...'`), and section 9.3 covers an admin who is locked out.

---

9. Security considerations

9.1 Threats and answers

Threat	Answer
Someone pretends to be a colleague and requests a reset, hoping the admin hands over a password	A request changes nothing. The admin must verify the person (in person, or a call back to a known number); the dialog says so. Requests show time, browser and address to help spot odd ones.
Guessing the temporary password	59 bits of randomness, 3-day expiry, the existing sign-in lock (5 tries per 15 minutes).
Temporary password read over a shoulder or kept in chat	One-time, expires, and useless after the first change; the first thing the person does is replace it.
Old password known to someone else	Reset and change both end every existing session (`session_version`).
Flagged person calls an action or API directly	Layer 2 refuses every action, page and route except the change itself.
Using a stolen signed-in session to change the password	Needs the current password, with the same lock as sign-in.
E-mail enumeration through the forgot form	Same answer and similar timing for every e-mail.
Spam through the request form	One open request per e-mail, 5 per hour per address, capped storage, purge after 30 days.
Admin misuse	Every reset is audited with who reset whom; admins cannot reset other admins or the superuser (only the superuser can reset an admin); nobody resets themselves from the panel.
A second superuser appears	The database allows one row with the superuser role, and the app cannot assign it.
Password in logs	Never logged, never in the audit trail, never in an error message; the one-time value exists only in the action's response.

9.2 What this does not cover

E-mail or SMS delivery of the temporary password (see section 12), two-step sign-in for admins, and password expiry after N days (not recommended: forced rotation without cause tends to weaken passwords).

9.3 If the superuser or every admin is locked out

Admins can be reset by the superuser, and the superuser's own sign-in problems (wrong password lock, forgotten password) are solved only by the developer's script, so it is worth keeping **one spare admin** account too; the superuser can reset it. For the case where none can sign in, `scripts/reset-password.ts` (run by the developer with the database connection, `yarn db:reset-password <email>`) generates a temporary password, sets the flag and prints the password once in the terminal. It writes an audit line with the actor "system". It is never exposed through the web.

---

10. Tests

Automatic (pure)

1. Password rules: length, 72-byte limit including multi-byte characters, equal to e-mail / name / current, common-password list, passphrase with spaces accepted.
2. Temporary password: length, alphabet (no look-alike characters), group format, uniqueness across 10,000 draws.
3. Expiry check and the allowed-path table for the proxy (a function of path, method and flag).
4. Request throttle rules (one open per e-mail, 5 per hour per address).

By hand, with a throw-away account

Check	Expected
Admin adds a user	A temporary password is shown once; closing the dialog and reopening the user does not show it
New user signs in with it	Lands on Change password, not the dashboard
Flagged user types `/cashflow`, `/cashflow/transactions`	Redirected to Change password
Flagged user calls an API route (`/api/cashflow/open-visits`)	403
Flagged user triggers an action with the browser tools	"Change your password first."
Wrong current password 5 times on the change page	Locked, same message as sign-in
New password equal to the temporary one, too short, common, or equal to the e-mail	Refused with a clear message
Valid new password	Menu opens; signing out and in with the new password works; the temporary one no longer works
Same user signed in on a second device, then changes password	The second device is signed out at its next click
Temporary password used on day 4	"This temporary password has expired."
Forgot password with a real e-mail and with a made-up one	Identical on-screen message; the admin sees one row "no matching account" for the made-up one
Request twice for the same e-mail	One open request
Admin presses Reset on the request	New temporary password shown once; the request moves to Done; the user's lock is cleared; the user's current session ends
Admin tries to reset own account from the panel	Refused with a pointer to Profile

Check	Expected
Admin tries to reset another admin, or the superuser	Refused (button hidden; direct action call also refused)
Superuser resets an admin	Works; temporary password shown once
Superuser tries to reset themselves	Refused with a pointer to Profile
Someone tries to create or promote a second superuser (screen or direct action call)	Refused; the database index also blocks it
Request for the superuser's e-mail	Same on-screen message; nothing appears in any admin's list
Request for an admin's e-mail	Admins see it read-only; the superuser can act
Staff user opens Users or tries the reset action	Redirected / refused
Audit tab	Lines for create, reset, change and request, none containing a password
Voluntary change from Profile	Works, does not set the flag, signs out other devices

11. Decisions to confirm

#	Question	Proposed default
D1	Should the admin still be able to **type** a password for a new user, or only generate one?	Generate only. It removes weak and reused passwords and the temporary one is replaced at once.
D2	How long does a temporary password last?	3 days; an admin can issue another at any time.
D3	Minimum password length for new passwords.	10 characters plus the common-password list (today 8).
D4	Store requests for e-mails that match no account?	Yes, tagged "no matching account", capped and purged after 30 days.
D5	Sign the person out of other devices after a change or reset?	Yes.
D6	**Existing accounts:** make everyone who never chose their own password (all of them today) change it at their next sign-in?	Yes, with one SQL line at rollout, except the person running the rollout. Delete or change the shared test account first.
D7	Who may reset an **admin's** password?	Only the superuser (replaces the earlier "admin re-enters their own password" idea). Admins reset dentists and staff.
D8	How are admins told of a new request?	The count in the menu and on the Users page for now; e-mail or chat alerts only if the clinic adds a mail service.
D9	**Which account is the superuser?** The existing test account, or a new account for the real owner?	A real person's account (the clinic owner or Kim's choice); retire the shared test account afterwards.
D10	Should a forgotten **superuser** password be reset only by the developer script?	Yes, since nobody is above the superuser.

12. Not in this plan, and how it could grow

- **E-mail or SMS reset links.** If the clinic later adds a mail service (Resend, Postmark, SES), the same request table and flag support a self-service path: the admin step becomes optional, a single-use hashed token with a 30-minute expiry is e-mailed, and the person lands on the same Change password page. The flag, the page, the session version and the audit lines do not change.
- **Two-step sign-in (authenticator app) for admins**, after this work.
- **A user "active" flag** (disable instead of delete) from the October review fits the same guard.
- **Reports on password hygiene** (for example "never changed" or "temporary password unused for 3 days") using the new columns.

Appendix A. Wording to change

Where	Now	Proposed
Forgot password page, intro	"we'll send you a link to reset your password"	"Enter your account e-mail. An admin will give you a temporary password after checking it is you."
Forgot password page, hero	"Reset links expire after a short window"	"Your admin issues a temporary password, and you choose your own at sign-in."
Forgot password, after sending	"you'll receive reset instructions shortly"	"Request sent. An admin will contact you. If it is urgent, ask your branch admin directly."
Button	"Send reset link"	"Request a reset"
Locked sign-in message	"Please wait 15 minutes..."	add "Forgot your password? Request a reset."
Get access page	"send you a temporary password"	keep; add "You will be asked to choose your own password the first time you sign in."
Change password page	new	"Welcome, (name). Choose your own password to continue."

As built (November 5, 2026)

All of phases 0 to 4 were built, using the default for every decision in section 11 (D1 generate-only, D2 3 days, D3 10 characters, D4 store unknown e-mails, D5 sign out other devices, D7 only the superuser resets admins, D8 menu count, D10 break-glass only for the superuser). D6 (force every existing account to change) is **not** applied automatically: it is a commented line at the end of the SQL file, to be switched on when the clinic decides. D9 (which account is the superuser) defaults to `superuser@admin.com` in the SQL file; change that e-mail before running it if another account should hold the role.

Differences from the plan, and why

Plan	As built	Reason
New `getActor()` guard from the permissions plan	The existing `auth()` that every page, action and API route already calls now lives in `src/lib/auth/access.ts` and asks the database each request (flag, session version, role)	One change covers all 34 call sites, with nothing to forget. The permissions work can build on it.
A fourth role in session and checks	In a session the superuser carries role `admin` plus `isSuperuser: true`	Every existing admin check keeps working unchanged; only the new who-may-reset-whom rule looks at the flag.

Plan	As built	Reason
Token flag refreshed by "update"	Same, plus a server-made proof (`adopt`) so only the person who just changed their password can carry their own browser forward	Without it, a browser could ask for the new session version and dodge the sign-out that follows a reset.
Role taken from the token	Role, name and e-mail now come from the database on every request	A demotion or removal takes effect on the next click, not at the next token refresh.
Admin approval to reset another admin	Only the superuser can; admins can still create, change and remove admins as before	Matches the request. Limiting admins further is listed in the security suggestions.

Files

New: `src/lib/auth/access.ts`, `password-policy.ts`, `temp-password.ts`, `reset-rules.ts`; `src/lib/db/password-requests.ts`; `src/app/change-password/` (page, actions); `src/app/forgot-password/actions.ts`; `src/app/session-ended/route.ts`; `src/components/auth/change-password-form.tsx`; `scripts/reset-password.ts` (`yarn db:reset-password <email>`); `scripts/sql/2026-11-password-management.sql`, `2026-11-superuser-role.sql`.

Changed: `src/auth.ts`, `src/proxy.ts`, `src/types/next-auth.d.ts`, `src/lib/db/schema.ts`, `src/lib/db/users.ts`, `src/lib/cashflow/user-schema.ts`, audit types, the Users page and its manager, the cashflow layout and menu, Profile, Forgot password, Get access and sign-in wording, and the import line in 34 files (`auth` now comes from `@/lib/auth/access`).

Safe before the SQL is run. Until the columns exist the app signs people in as before, nobody is forced to change a password, the Users page and Profile still open, and **Add user / Reset password** show "Run scripts/sql/2026-11-password-management.sql first". Checked on the live development database before any change was made to it.

Run order

1. Run `2026-11-password-management.sql` in the Neon SQL editor.
2. Run `2026-11-superuser-role.sql` as two separate runs (step 1, then step 2; the file explains why). Change the e-mail in step 2 first if needed.
3. Deploy / restart the app. Make sure `AUTH_SECRET` is set (it already is for sign-in).
4. Sign in as the superuser and add a throw-away account to see the flow.

Tested

- Type check, lint and a production build: clean.
- 20 automatic checks of the password rules, temporary passwords and who-may-reset-whom table.
- 73 browser checks on a private test database (not the clinic's data) covering sections 4 and 10 of this document: creating an account, one-time display of the temporary password, forced change from every page, refused API and form calls while flagged, wrong, short, common, reused and mismatched passwords, the change lock, sign-out of other devices, forgot-password requests (same answer for known, unknown, repeated, admin and superuser e-mails), admin reset from a request, superuser reset of an admin, admins not being able to reset admins, dismiss, expired temporary password, signed-out access, and the one-superuser rule. 71 passed on the first run; the other two were faults in the test itself (a missing wait, and a database index that the test database lacked), both rechecked and passing.

- The two SQL files were run on a copy of the old schema (and run twice) to confirm they work as written.
- Break-glass script checked on the test database.
- Sign-in, the 34 changed pages and the API routes loaded on the clinic's development server without the SQL (the fallback path).

Not tested: e-mailed notifications (none exist), the "Copy" buttons' clipboard access (browser dependent), phone layout of the new dialog, and a run against the clinic's real Neon database after the SQL.